Expense Approval System

A multi-database application for auditable decisions and semantic retrieval

PROJECT SUMMARY

This Python application decides whether an employee expense should be **approved, rejected, or escalated**. It separates financial state, receipt evidence, delegated authority, short-lived caches, and semantic retrieval across five database models. The implementation uses PostgreSQL and MongoDB as authoritative stores; Redis, Neo4j, and vector indexes provide derived or specialized access paths. The evaluation uses generated synthetic records and compares pgvector with Qdrant on the same text embeddings.

1 System architecture and data ownership

Technology	Data and role	Reason for using it
PostgreSQL + pgvector	Employees, cost centres, budgets, expenses, decisions; optional vector search	Transactions and joins; pgvector supports similarity alongside relational data.
MongoDB	Receipts and vendor/country-specific fields	Document structure with enforced JSON Schema; source of receipt evidence.
Redis	Approver-chain cache with 300-second TTL	Fast temporary lookups and explicit invalidation when delegations change.
Neo4j	Employees, cost-centre ownership, conditional DELEGATED_TO edges	Variable-length path checks across scoped delegation relationships.
Qdrant	Receipt and policy embeddings with payload metadata	Dedicated vector retrieval with payload filtering for comparison against pgvector.

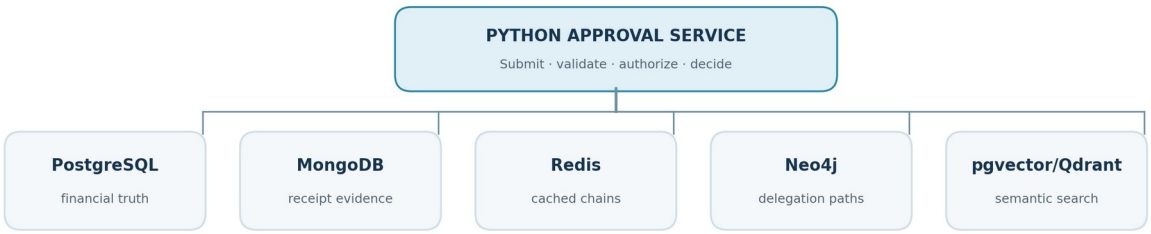


Figure 1. Logical roles around the Python approval workflow. pgvector runs inside PostgreSQL; Qdrant is a separate service.

Decision path

A submitted expense is linked to receipt evidence; the service checks financial state, resolves candidate approvers, verifies live delegation authority, and records the outcome. An approval updates its budget and decision together, while semantic receipt matching and policy lookup provide supporting analysis rather than replacing the authorization decision.

Seeded data and provenance

The synthetic test environment contains about **253 employees, 252 expenses, 10 cost centres, 10 budgets, and 252 MongoDB receipts**. The graph includes synthetic cost-centre responsibilities and delegated approval edges, while the vector corpus contains 455 receipt and policy texts. The records are generated for coursework evaluation, not collected from a production financial system.

2 Transaction integrity, documents and authorization

PostgreSQL: atomic financial decisions

The approval service locks relevant rows and checks the budget balance and expected version before updating it. Budget reduction, version increment and the approval decision are committed in one PostgreSQL transaction. A version conflict or insufficient balance prevents an unsafe approval; conflict-specific retry logic exists for the integrated workflow.

Observed concurrency case. With a €1,000 budget and two pending €700 expenses, one approval succeeded, leaving €300 and incrementing the version to 2. The competing approval failed its budget/version check; it remained pending, so the budget was not overspent.

MongoDB: validated receipt evidence

Receipts store vendor and invoice details, dates, country, total, and VAT-related fields, using Decimal128 for monetary values. MongoDB's JSON Schema validator rejects documents missing required fields such as total or VAT number. PostgreSQL relates business expenses to MongoDB receipt identifiers without duplicating the entire document as relational columns.

Redis: performance without trusting cached permissions

Approver chains are cached under keys such as **approver_chain:<employee_id>** with a **300-second TTL**. A cached chain avoids repeated graph lookups, but it can become stale when a delegation is revoked. The implementation demonstrates stale cache state and key deletion/refresh; final approval authorization is still checked against Neo4j. Thus Redis accelerates lookup but is not an authoritative permission store.

Neo4j: bounded, constrained delegation paths

A cost-centre owner has a **RESPONSIBLE_FOR** relationship to a CostCentre. Delegation edges between employees are labelled **DELEGATED_TO** and carry **scope, amount_limit, valid_from** and **valid_to**. Authorization succeeds only if a permitted path connects the owner to the acting approver and every edge satisfies the expense's scope, amount and date.

```
MATCH (owner:Employee)-[:RESPONSIBLE_FOR]->(cc:CostCentre)
MATCH path = (owner)-[:DELEGATED_TO*0..5]->(approver:Employee)
WHERE ... AND ALL(rel IN relationships(path) WHERE
  rel.scope = cc.name AND rel.amount_limit >= $amount
  AND date(rel.valid_from) <= date($expense_date)
  AND date(rel.valid_to)   >= date($expense_date))
```

Excerpt of the authorization logic in app/services/authorization_service.py; omitted parameters and final return are represented by ellipsis.

The application bounds traversal at **0–5 delegation hops**: zero hops authorizes the responsible owner, while one to five edges permit delegated approval. The hop bound is an application choice, not a Neo4j-wide maximum.

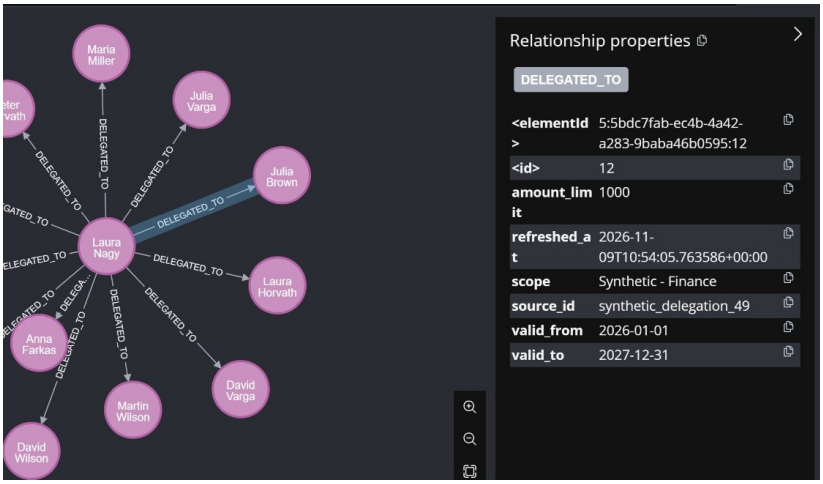


Figure 2. Neo4j Browser: synthetic delegations and the selected edge's conditions.

3 Vector retrieval and measured results

Receipt texts and expense-policy passages are encoded with **all-MiniLM-L6-v2** into normalized **384-dimensional** vectors. The same document IDs and cosine-based embeddings are stored in pgvector and Qdrant, together with document type, source identifier, and refresh metadata. The development corpus contains **455 vectors** (205 receipt/policy examples plus 250 receipt records derived from MongoDB). Qdrant payload indexes support document_type and filter_group filtering, and PostgreSQL has a cosine HNSW vector index.

Relational lookup difference. With pgvector, vector matches can be joined to expense records in one SQL statement through the receipt-to-expense mapping. With Qdrant, the service first retrieves matching document identifiers and then performs a second-stage PostgreSQL lookup for business data.

Labelled retrieval tasks

Evaluation	Measured outcome	Interpretation
Expense decisions (25 cases)	25/25 in a simplified rules evaluator	Logic-level cases; not full end-to-end database transactions.
Duplicate receipts (25 pairs)	Cosine threshold: P=0.769, R=1.000; hybrid: P=1.000, R=1.000	Hybrid also checks invoice/vendor; exploratory pair set, not a held-out test.
Policy retrieval (20 queries)	Recall@5 = 1.000; MRR = 0.975	Five repeated variants per policy were collapsed to canonical groups.
Unfiltered PostgreSQL ANN (20 queries)	HNSW ef_search=40: recall@10 = 1.000	Verified against exact results with separate transaction settings and plan checks.

Rare-filter benchmark (20 fixed queries)

To test selective filtering, **23 of 455 vectors (5.05%)** were assigned the same rare label in both stores. Each query requested ten results; both filtered exact-search baselines returned ten identical document IDs.

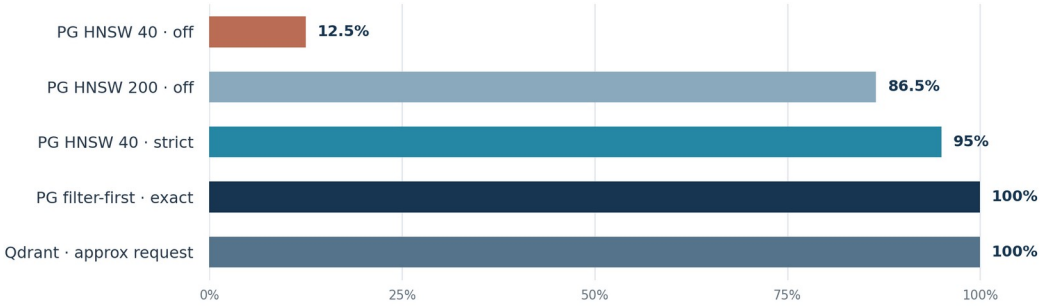


Figure 3. Mean filtered recall@10. "Strict" means PostgreSQL HNSW iterative_scan=strict_order; "off" means no iterative scanning. Qdrant was requested with exact=False, hnsw_ef=40.

Search configuration	Recall@10	Mean returned	Mean latency
PG exact / materialized filter	1.000	10.00	51.054 ms
PG HNSW ef=40, iterative off	0.125	1.25	51.933 ms
PG HNSW ef=200, iterative off	0.865	8.65	51.523 ms
PG HNSW ef=40, strict	0.950	10.00	48.599 ms
Qdrant exact	1.000	10.00	14.891 ms
Qdrant approx requested, hnsw_ef=40	1.000	10.00	14.037 ms

4 Failure case, design lessons and reproducibility

Failure case: ANN search with a rare metadata filter

With only 5.05% of documents eligible, PostgreSQL HNSW at **ef_search=40** and **iterative_scan=off** returned just **1.25 of ten** requested neighbors on average, giving **12.5% mean recall@10**. The index was traversed in vector order and the SQL filter discarded many candidates. This is a completeness failure despite ten or more eligible documents existing.

Increasing ef_search to 200 raised mean recall to **86.5%** but still returned fewer than ten results for nine queries. At ef_search=40, **strict iterative scanning raised recall to 95%** and returned ten results on every query, though four queries still missed at least one exact neighbor. A natural PostgreSQL query plan instead used its conventional **filter_group index + exact distance sort** and returned ten for every query (48.851 ms mean client-observed latency).

Qdrant returned ten of ten neighbors at 100% recall under its approximate-search request. For such a small filtered subset, Qdrant may internally select an exact scan; these numbers therefore compare **requested search configurations**, not proven HNSW traversal in both systems. Latency differences include client/network overhead and should not be generalized from this small synthetic dataset.

Derived state and consistency boundaries

PostgreSQL is authoritative for money and decisions; MongoDB is authoritative for receipt evidence. Redis entries, Neo4j relationship projections and vectors are derived views with different refresh and invalidation mechanisms. The project demonstrates stale cache and vector state followed by refresh, while the final authorization path consults the current delegation graph. There is no single atomic transaction spanning all stores, so change propagation and recovery remain explicit responsibilities.

Reproduction notes

The original experiments used Windows, Docker Compose and Python 3.11. Services: **pgvector/pgvector:pg16**, **mongo:7.0.16**, **redis:7.4.2**, **neo4j:5.26.2-community**, and **qdrant/qdrant:v1.13.3**. Relevant laptop dependencies include psycopg 3.3.6, qdrant-client 1.13.2, sentence-transformers 6.2.0, and torch 2.15.0; the full Python environment is captured in requirements.lock.txt.

The repository includes deterministic benchmark queries and labelled fixtures (for example, **data/duplicate_test_cases.json** and **data/policy_test_cases.json**), alongside evaluation scripts. After starting the configured services and initializing/seeding the databases, key checks include **app.verify_postgres_hnsw_recall**, **app.benchmark_filtered_search**, **app.evaluate_policy_retrieval** and **app.evaluate_hybrid_duplicates**. Do not recreate populated volumes when reproducing a completed run.

Evidence and verification checkpoints

Question	Demonstration or test
Can concurrent approvals overspend?	Budget version conflict test; one €700 approval succeeds, competing approval fails.
Can malformed receipts be stored?	MongoDB JSON Schema rejects missing total or VAT number.
What if cached authorization changes?	Redis chain TTL, simulated stale cache, deletion, and live Neo4j re-check.
How is delegation evaluated?	Bounded Cypher traversal *0..5 with per-edge scope, amount, and date predicates.
Do vector retrieval modes differ?	Exact/HNSW and filtered evaluations against fixed query sets and exact baselines.

Limitations and conclusion

The dataset and labels are synthetic. The hybrid duplicate rules were assessed on an exploratory set, not an independent hold-out; the 25 expense scenarios do not validate the full integrated workflow; and large-scale throughput, crash recovery and automated cross-store reconciliation were not evaluated. Even within these limits, the project illustrates a practical division of responsibility: **commit money atomically, validate evidence, cache selectively, authorize from live graph relationships, and treat vector search as supporting evidence.**